- 1. 流水线组织
- 2. 指令执行路径
- 3. 数据冒险与前递
  - 3.1 前递网络
  - 3.2 前级停顿框架
- 4. M 扩展状态机
  - 4.1 乘法
  - 4.2 除法与取余
- 5. Difftest 对拍时序
- 6. 性能分析
- 7. AI 使用说明

Lab1 实现说明

1. 流水线组织

实现使用 5 级顺序流水，核心流水寄存器为：

- `id_r`: `valid/pc/instr`
- `ex_r`: `valid/trap/wen/is_word/alu_cmd/rd/pc/instr/op1/op2`
- `mem_r`、`wb_r`: `valid/trap/wen/rd/pc/instr/result`

更新顺序为 `wb<=mem<=ex<=id`。`x0` 每拍强制写 0，防止误写路径污染状态。

2. 指令执行路径

支持的 Lab1 指令在 Decode 中统一解码为 `alu_cmd + op1/op2 + rd/wen/is_word`：

- I 型：`addi/xori/ori/andi`
- R 型：`add/sub/xor/or/and`
- W 型：`addiw/addw/subw`
- M 扩展：`mul/div/divu/rem/remu`、`mulw/divw/divuw/remw/remuw`

基础 ALU（add/sub/xor/or/and）组合完成；`is_word=1` 时对低 32 位结果做符号扩展。

3. 数据冒险与前递

3.1 前递网络

Decode 阶段先读寄存器文件，再做组合覆盖，优先级为 `EX > MEM > WB > GPR`：

- `id_rs*_val` 默认来自 `gpr[rs*]`
- 命中 EX 且可前递则取 `ex_result`
- 否则命中 MEM 取 `mem_r.result`
- 否则命中 WB 取 `wb_r.result`

EX 可前递条件：

- `ex_forwardable = ex_r.valid && ex_r.wen && (ex_r.rd != 0) && ex_result_ready`

其中 `ex_result_ready = !ex_is_mdu || mdu_out_valid`，保证多周期 MDU 未完成时不会前递中间值。

3.2 前级停顿框架

当前实现把“前级是否冻结”统一收敛到：

- `stall_front = stall_ex_busy || raw_hazard_ex || raw_hazard_mem`

定义如下：

- `stall_ex_busy`: EX 指令结果尚未就绪（当前主要对应 MDU 进行中）
- `raw_hazard_ex`: 与 EX 未就绪结果形成 RAW
- `raw_hazard_mem`: 预留给后续 Lab 的 MEM 未就绪 RAW（当前 `mem_result_ready=1`）

取指与前级推进都受 `stall_front` 控制：

- 前端采用“单请求 in-flight + 一拍取指缓冲”的 fetch FSM：
  - 发起请求后保持 `ireq.valid/ireq.addr` 直到 `iresp.data_ok`
  - 响应先写入 `fetch_buf`，前端解冻后再消费（`fetch_fire`）送入 `id_r`
  - `stall_front` 仅阻止新请求与缓冲消费，不会修改 in-flight 请求

这样可以直接扩展到后续的 load-use 冒险：只需把 `mem_result_ready` 接到真实访存返回时序，现有框架即可复用。 同时也避免了在 ibus 多拍返回场景下“等待 `data_ok` 期间请求被改写”的时序风险。

4. M 扩展状态机

4.1 乘法

`mul/mulw` 使用移位加法迭代器，每拍处理 1 bit：

- 若乘数最低位为 1，累加被乘数
- 被乘数左移、乘数右移
- 计数减 1，结束时输出

4.2 除法与取余

`div/divu/rem/remu`（含 W 变体）使用恢复除法迭代：

- 余数左移并引入被除数最高位
- 与除数比较，满足则减法并置商位
- 迭代结束后按有符号/无符号规则修正商或余数符号

边界条件在启动阶段处理：

- 除零：`div/divu -> 全 1`，`rem/remu -> 被除数`
- 溢出：`MIN_INT / -1` 返回 `MIN_INT`，余数返回 0

W 指令迭代步数固定为 32，并在输出时做 32->64 符号扩展。

在此基础上做了 3 组性能优化，减少无效迭代：

- `mul/mulw`：当乘数高位剩余部分全 0 时提前结束
- `div/rem` 快速路径：
  - `divisor == 1` 直接出结果
  - `dividend < divisor` 直接返回商 0 / 余数被除数
- `div/rem` 动态步数：按被除数和除数有效位计算 `steps = bits(dividend) - bits(divisor) + 1`，不再固定跑满 64/32 轮

5. Difftest 对拍时序

- `DifftestInstrCommit` 从 WB 提交
- `wdest` 使用 `{3'd0, wb_r.rd}` 做 5->8 位扩展
- `DifftestTrapEvent.code` 使用 3 位 `trap_code_latched`

寄存器对拍使用 `gpr_diff`：

- 先复制 `gpr`
- 若 WB 当拍写回则覆盖 `gpr_diff[rd]`

这样 Difftest 看到的是“提交当拍已生效”的架构态，避免 WB 与对拍时序错拍。

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x0000000080010004
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 2179 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 16385
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 7519504 instr/s
Core 0: HIT GOOD TRAP at pc = 0x80010004
total guest instructions = 16386
instrCnt = 16386, cycleCnt = 32775, IPC = 0.499954
Seed=0 Guest cycle spent: 32778 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 59ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use +jtag_rbb_enable=1.
Listening on port 23334
xy@Archer:/mnt/g/Github/26-Arch$
```

```
The first instruction of core 0 has commited. Difftest enabled.
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008002001c
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 4842 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32775
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 6768897 instr/s
Core 0: HIT GOOD TRAP at pc = 0x8002001c
total guest instructions = 32776
instrCnt = 32776, cycleCnt = 774846, IPC = 0.042300
Seed=0 Guest cycle spent: 774849 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 159ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use +jtag_rbb_enable=1.
Listening on port 23334
xy@Archer:/mnt/g/Github/26-Arch$
```

6. 性能分析

已观测到 `test-lab1` IPC 明显高于 `test-lab1-extra`。原因是：

- `test-lab1` 以单周期基础 ALU 为主，流水线接近稳态
- `test-lab1-extra` 含大量 M 指令，MDU 多周期执行拉长 EX 占用时间

这不是功能错误，而是当前微结构选择（多周期 MDU + 顺序单发射）的直接结果。本实现已做的优化点包括：

- W 类指令用 32 位语义执行并符号扩展
- `mul` 快速路径与提前终止（`0/1` 快返、乘数剩余位全 0）
- `div/rem` 快速路径（`dividend=0`、`divisor=1`、`dividend<divisor`、`divisor` 为 2 的幂）

补充说明（按优化前后 IPC 对比）：

- 优化前：`test-lab1` IPC 约 `0.49`，`test-lab1-extra` IPC 约 `0.0423`，两者约 `11.6x`，接近一个数量级差距。
- 优化后（`-j12` 实测）：
  - `test-lab1`：`instrCnt=16386, cycleCnt=32776, IPC=0.499939`
  - `test-lab1-extra`：`instrCnt=32776, cycleCnt=246683, IPC=0.132867`
- 优化后两者 IPC 比值约为 `0.499939 / 0.132867 ≈ 3.76x`，约 3 倍多。

后续若继续提升 IPC，可在不改 ISA 可见行为的前提下做两类优化：

- 更快迭代器（如 radix-4/Booth）
- 细化停顿条件（让与长延迟结果无关的路径减少等待）

```
[src/cpu/cpu-exec.c:393,cpu_exec] nemu: HIT GOOD TRAP at pc = 0x000000008002001c
[src/cpu/cpu-exec.c:394,cpu_exec] trap code:0
[src/cpu/cpu-exec.c:74,monitor_statistic] host time spent = 4214 us
[src/cpu/cpu-exec.c:76,monitor_statistic] total guest instructions = 32775
[src/cpu/cpu-exec.c:77,monitor_statistic] simulation frequency = 7777645 instr/s
Core 0: HIT GOOD TRAP at pc = 0x8002001c
total guest instructions = 32776
instrCnt = 32776, cycleCnt = 246682, IPC = 0.132867
Seed=0 Guest cycle spent: 246685 (this will be different from cycleCnt if emu loads a snapshot)
Host time spent: 86ms
This emulator compiled with JTAG Remote Bitbang client. To enable, use +jtag_rbb_enable=1.
Listening on port 23334
```

7. AI 使用说明

大模型用于：

- 帮助整理实现方案和报告结构
- 检查位宽、接口连线和边界条件覆盖清单
- 生成回归验证步骤

最终 RTL 设计、时序决策、调试定位与测试结论由本人完成。